24CS01EF - Python Full Stack Development

Lab Workbook

Python Full Stack Development Team
Department of Computer Science & Engineering,
KLEF (Deemed to be University), Green Fields, Vaddeswaram, Andhra Pradesh

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Accredited by NAAC as 'A++' Grade University ❖Approved by AICTE ❖ ISO 9001-2015 Certified
Campus: Green Fields, Vaddeswaram - 522 502, Guntur District, Andhra Pradesh, INDIA.
Phone No. 0863 - 2399999; www.klef.ac.in; www.klef.edu.in; www.kluniversity.in
Admin Off: 29-36-38, Museum Road, Governorpet, Vijayawada - 520 002. Ph: +91 - 866 -2577715, Fax: +91-866-2577717.

24CS01EF - Python Full Stack Development

Lab Workbook

STUDENT NAME	
REG NO	
YEAR	
SEMESTER	
SECTION	
FACULTY NAME	

INDEX

Sl.No	Date of Execution	Title	Program / Write – up [10]	Execution & Output [40]	Total [50]	Faculty Sign
1		Python Basics – Variables, Data Types, and Control Statements				
2		Simple Calculator Using Python				
3		Python Functions and Modules				
4		File Handling and Exception Handling				
5		Object-Oriented Programming in Python				
6		Data Analysis with Pandas				
7		Creating a Basic Django Project with URL Mapping and Views				
8		Student Profile Management Application Using Django				
9		Student Feedback Collection Application Using Django with PostgreSQL				
10		Weather Application Using Django				
11		Student Management System Using Django ORM with PostgreSQL (CRUD Operations)				
12		Creating a Flask Application with Routes, Templates, and User Input				
13		Flask API Development				
14		Using Python Libraries with Flask				
15		Deployment of Django Application Using PythonAnywhere				

KL University Vision and Mission

Vision :

To be a globally renowned university.

Mission :

To impart quality higher education and to undertake research and extension with emphasis on application and innovation that cater to the emerging societal needs through all-round development of the students of all sections enabling them to be globally competitive and socially responsible citizens with intrinsic values.

Table of Contents

Date of Execution:

Experiment 1: Python Basics – Variables, Data Types, and Control Statements

Scenario**:**

You are developing a beginner-friendly Python utility program for first-year students to understand **Python syntax, variables, data types, and conditional statements** through a real-world student data example.

Tasks:

- Read a number from the user.
- Check whether the number is **positive, negative, or zero** using conditional statements.
- Create variables to store **student ID, student name, marks, and attendance status** using appropriate data types.
- Display the **value and data type** of each variable.
- Perform basic arithmetic operations on numeric values (addition, subtraction, multiplication, division).

Expected Output:

- The program should correctly identify whether the entered number is positive, negative, or zero.
- Student details should be displayed along with their respective data types.
- Results of arithmetic operations should be shown correctly.

Viva Questions:

1. What are variables in Python and how are they created?
2. What are the built-in data types in Python?
3. How does Python handle conditional statements?
4. What is the difference between '=' and '=='?
5. How does Python determine the data type of a variable?

	Marks Secured:
Evaluator Remark (if Any):	**Signature of the Evaluator with Date**

Date of Execution:

Experiment 2: Simple Calculator Using Python

Scenario:

You are asked to develop a simple calculator application to help students understand **user input, operators, and conditional logic** in Python.

Tasks:

- Read two numbers from the user.
- Display a menu with arithmetic operations:
  - Addition
  - Subtraction
  - Multiplication
  - Division
- Allow the user to select an operation.
- Perform the selected operation and display the result.
- Handle invalid inputs and division by zero using exception handling.

Expected Output:

- The calculator should perform the selected arithmetic operation correctly.
- Appropriate error messages should be displayed for invalid input or division by zero.

Viva Questions:

1. What are arithmetic operators in Python?
2. How do conditional statements help in menu-driven programs?
3. What is exception handling in Python?
4. Why is division by zero an exception?
5. What is the use of try and except blocks?

	Marks Secured:
Evaluator Remark (if Any):	**Signature of the Evaluator with Date**

Date of Execution:

Experiment 3: Python Functions and Modules

Scenario:

In an academic system, student grades are calculated repeatedly based on their marks in different subjects. To avoid rewriting the same logic multiple times, you are required to develop a **reusable grading functionality** using Python **functions and modules**. This helps students understand how to write modular code and reuse it across different programs.

Tasks:

- Design a Python function that accepts student marks as input and calculates the corresponding grade based on predefined grading criteria.
- Save the grade-calculation function in a **separate Python module** to promote code reusability.
- Import the custom module into a main Python program.
- Call the grading function from the main program for different student marks.
- Display the calculated grade for each input clearly.

Expected Output:

- The grading function should correctly determine grades for various mark ranges.
- The function should be reusable and callable from different programs.
- The main program should successfully import the module and display the correct grade for each student input.

Viva Questions:

1. What is a function in Python?
2. Why are functions considered reusable code?
3. What is a module in Python?
4. How do you import a custom module?
5. What are the advantages of modular programming?

	Marks Secured:
Evaluator Remark (if Any):	**Signature of the Evaluator with Date**

Date of Execution:

Experiment 4: File Handling and Exception Handling

Scenario:

Student attendance data needs to be saved and retrieved from a file securely.

Tasks:

- Write student data to a text file.
- Read and display the file contents.
- Handle exceptions such as file not found and invalid input.

Expected Output:

- Data should be written and read successfully.
- Proper error messages should be displayed during exceptions.

Viva Questions:

1. What are file modes in Python?
2. What is the difference between read and write modes?
3. What is exception handling?
4. What happens if a file does not exist while reading?
5. Why is file handling important in real-world applications?

	Marks Secured:
Evaluator Remark (if Any):	**Signature of the Evaluator with Date**

Date of Execution:

Experiment 5: Object-Oriented Programming in Python

Scenario:

Design a system to manage student details using object-oriented principles.

Tasks:

- Create a Student class with attributes and methods.
- Implement inheritance using a GraduateStudent class.
- Demonstrate encapsulation and polymorphism.

Expected Output:

- Student details should be displayed correctly using class objects.

Viva Questions:

1. What is object-oriented programming?
2. What is a class and an object?
3. What is inheritance in Python?
4. What is encapsulation?
5. What is polymorphism?

	Marks Secured:
Evaluator Remark (if Any):	**Signature of the Evaluator with Date**

Date of Execution:

Experiment 6: Data Analysis with Pandas

Scenario:

Alex is a data analyst who needs to perform basic data analysis on a CSV file containing sales data. The dataset includes information such as date, product category, and sales amount. Alex wants to analyze the data efficiently using the **Pandas** library.

Tasks:

- Load a CSV file containing sales data using Pandas.
- Handle missing or corrupt data gracefully (using appropriate Pandas methods).
- Calculate the **total sales** and **average sales** from the dataset.
- Filter the sales data for a **specific month** provided by the user.
- Group the data by **product category** and calculate **total sales for each category**.
- Display the results in a clear and readable format.

Expected Output:

- The CSV file should be loaded successfully, even if it contains missing values.
- Total sales and average sales should be calculated correctly.
- Filtered data for the selected month should be displayed.
- Total sales grouped by product category should be shown accurately.

Viva Questions:

1. What is Pandas and why is it used?
2. What is a DataFrame?
3. How does Pandas handle missing data?
4. What is grouping in Pandas?
5. What is the difference between sum() and mean()?

	Marks Secured:
Evaluator Remark (if Any):	**Signature of the Evaluator with Date**

Date of Execution:

Experiment 7: Creating a Basic Django Project with URL Mapping and Views

Scenario:

A university wants to develop a basic web portal using Django to display information related to **departments, courses, faculty, and students**. The portal should demonstrate the creation of a Django project, application setup, and navigation across multiple pages using URL mapping and views.

Tasks:

- Create a Django project and a Django app.
- Configure essential project settings (installed apps, templates, and static files).
- Create multiple Django views to display:
    - Home / Welcome page
    - Courses page
    - Faculty page
    - Students page
- Map URLs to their respective views using Django URL configuration.
- Display appropriate messages or content for each page.

Expected Output:

- The Django project should run successfully on the development server.
- The home page should display a welcome message.
- Each URL (home, courses, faculty, students) should load the correct page with appropriate content.

Viva Questions:

1. What is Django?
2. What is the role of views in Django?
3. What is URL mapping?
4. What is the purpose of urls.py?
5. How does Django follow the MVT architecture?

	Marks Secured:
Evaluator Remark (if Any):	**Signature of the Evaluator with Date**

Date of Execution:

Experiment 8: Student Profile Management Application Using Django

Scenario:

A university wants a simple web application that allows students to **view their profile information** such as name, roll number, department, and year of study. The application should present the data using a clean and consistent web interface.

Tasks:

- Create a Django project and app for student profile management.
- Design a common layout (header, footer, navigation) for all pages.
- Create views to display:
  - Student profile page
  - Academic details page
- Pass student information dynamically from Django views to templates.
- Render the data using Django templates.

Expected Output:

- The application should display student profile and academic details.
- All pages should share a common layout and navigation.
- Dynamic student data should be rendered correctly on the web pages.

Viva Questions:

1. What are templates in Django?
2. How is dynamic data passed to templates?
3. What is template inheritance?
4. What is the role of context data?
5. Why is a common layout used in web applications?

	Marks Secured:
Evaluator Remark (if Any):	**Signature of the Evaluator with Date**

Date of Execution:

Experiment 9: Student Feedback Collection Application Using Django with PostgreSQL

Scenario:

The university needs an online system to **collect and manage feedback from students** regarding courses or faculty. The application should accept user input through a web form, validate the data, and store the feedback securely in a **PostgreSQL database**. After submission, the user should receive a confirmation message.

Tasks:

- Create a Django project and a Django app for feedback collection.
- Configure **PostgreSQL** as the backend database in Django settings.
- Design a web form to collect the following details:
    - Student name
    - Course name
    - Feedback comments
    - Rating
- Implement server-side validation for form inputs.
- Create a Django model to store feedback details in the PostgreSQL database.
- Save the submitted feedback into the database using Django ORM.
- Display a confirmation or summary page after successful submission.

Expected Output:

- A feedback form should be displayed on the web page.
- User input should be validated properly before submission.
- Submitted feedback details should be stored successfully in the PostgreSQL database.
- A confirmation or summary message should be displayed after successful submission.

Viva Questions:

1. What is PostgreSQL?
2. Why use PostgreSQL with Django?
3. What is Django ORM?
4. How is form data validated in Django?
5. How is data stored securely in a database?

	Marks Secured:
Evaluator Remark (if Any):	**Signature of the Evaluator with Date**

Date of Execution:

Experiment 10: Weather Application Using Django

Scenario:

A university plans to build a simple **weather information web application** that allows users to check the current weather of a city. The application should fetch real-time weather data from an external API and display it using the Django framework.

Tasks:

- Create a Django project and a Django app for the weather application.
- Design a user interface with an input form to enter the city name.
- Integrate a public weather API to fetch current weather details.
- Handle user input and API responses in Django views.
- Display weather information such as:
  - City name
  - Temperature
  - Weather condition
  - Humidity
- Handle invalid city names or API errors gracefully.

Expected Output:

- A Django web page with a city input form.
- Upon submitting a valid city name, the current weather details should be displayed.
- Proper error messages should appear for invalid inputs or unavailable data.

Viva Questions:

1. What is an API?
2. What is a REST API?
3. How does Django fetch data from an external API?
4. What is JSON format?
5. Why is error handling important in API-based applications?

	Marks Secured:
Evaluator Remark (if Any):	**Signature of the Evaluator with Date**

Date of Execution:

Experiment 11: Student Management System Using Django ORM with PostgreSQL (CRUD Operations)

Scenario:

A university portal needs a **student management system** to store, update, view, and delete student records. The application should use **Django ORM** to perform complete **CRUD (Create, Read, Update, Delete)** operations with a **PostgreSQL database**.

Tasks:

- Create a Django project and a Django app for student management.
- Configure **PostgreSQL** as the backend database in Django settings.
- Design a Django model to store student information such as:
    - Student ID
    - Student Name
    - Department
    - Year of Study
    - Email ID
- Implement **CRUD operations** using Django ORM:
    - **Create:** Add new student records
    - **Read:** Display all student records on a web page
    - **Update:** Edit existing student details
    - **Delete:** Remove student records
- Design HTML templates to perform CRUD operations using forms.
- Validate user inputs before saving data.

Expected Output:

- Student records should be stored successfully in the PostgreSQL database.
- All existing student records should be displayed on a web page.
- Student details should be updated correctly.
- Student records should be deleted successfully.
- Database operations should work smoothly using Django ORM.

Viva Questions:

1. What does CRUD stand for?
2. What is Django ORM?
3. How does Django interact with PostgreSQL?
4. What is a model in Django?
5. What is the advantage of ORM over raw SQL?

	Marks Secured:
Evaluator Remark (if Any):	**Signature of the Evaluator with Date**

Date of Execution:

Experiment 12: Creating a Flask Application with Routes, Templates, and User Input

Scenario:

A lightweight Flask-based web application is required to display **department details** and interact with users by accepting input and displaying personalized responses. This demonstrates the use of **Flask routing, views, templates, and form handling**.

Tasks:

- Create a basic Flask application.
- Define multiple routes and views to:
  - Display department information
  - Display a user interaction page
- Create HTML templates using **Jinja2**.
- Design a form to accept the user's name as input.
- Process user input in Flask views.
- Display a personalized greeting message along with department details.
- Run the Flask development server and test all routes.

Expected Output:

- The Flask application should run successfully on the local server.
- A web page should display department information.
- A form should accept the user's name.
- Upon submission, a personalized greeting message should be displayed correctly.

Viva Questions:

1. What is Flask?
2. What is routing in Flask?
3. What is Jinja2?
4. How does Flask handle form data?
5. What is the difference between Flask and Django?

	Marks Secured:
Evaluator Remark (if Any):	**Signature of the Evaluator with Date**

Date of Execution:

Experiment 13: Flask API Development

Scenario:

A REST API is required to provide student data in JSON format.

Tasks:

- Create a Flask API endpoint.
- Return student details as JSON.
- Test the API using a browser or Postman.

Expected Output:

- JSON response containing student data.

Viva Questions:

1. What is a REST API?
2. What is JSON?
3. How do you create an API endpoint in Flask?
4. How can APIs be tested?
5. What is the difference between API and web application?

	Marks Secured:
Evaluator Remark (if Any):	**Signature of the Evaluator with Date**

Date of Execution:

Experiment 14: Using Python Libraries with Flask

Scenario:

A web app needs to visualize student performance statistics.

Tasks:

- Use Pandas to process student data.
- Use Matplotlib to generate a chart.
- Display the chart through a Flask application.

Expected Output:

- Graphical representation of student performance displayed on the web page.

Viva Questions:

1. What is Matplotlib used for?
2. How does Pandas help in data processing?
3. How can charts be displayed in a web application?
4. What type of data visualization is commonly used?
5. Why is data visualization important?

	Marks Secured:
Evaluator Remark (if Any):	**Signature of the Evaluator with Date**

Date of Execution:

Experiment 15: Deployment of Django Application Using PythonAnywhere

Scenario:

The university wants to deploy its Django web application for **public access** so that students and faculty can access it through the internet. The application should be deployed on a cloud-based platform using **PythonAnywhere**, which supports hosting Django applications.

Tasks:

- Create an account on **PythonAnywhere**.
- Upload the Django project to the PythonAnywhere server.
- Configure Django settings for production, including:
  - Configuring ALLOWED_HOSTS
  - Setting up static files
- Create and activate a virtual environment on PythonAnywhere.
- Install required dependencies for the Django project.
- Configure the **WSGI file** to link the Django project with PythonAnywhere.
- Run database migrations and collect static files.
- Deploy the Django application and make it accessible via a public URL.
- Test the deployed application through a web browser.

Expected Output:

- The Django application should be successfully deployed on PythonAnywhere.
- The application should be accessible through a **public PythonAnywhere URL**.
- All pages of the Django application should load correctly without errors.

Viva Questions:

1. What is deployment?
2. What is the difference between development and production settings?
3. What is a web server?
4. Why is security important in deployed applications?
5. What challenges are faced during deployment?

	Marks Secured:
Evaluator Remark (if Any):	**Signature of the Evaluator with Date**